



Risoluzione esercizio 2 - Architettura CPU RISC-V

Modifiche dell'architettura della CPU RISC-V e segnali di controllo

1. Analizza l'istruzione, come operare e scrivi le operazioni hardware da effettuare.

Un esempio pratico è il seguente:

1. **Lettura:** Leggi `rs1` dal banco registri ed estendi l'immediato (`immediate32`).
 2. **Calcolo Somma:** Somma `rs1 + immediate32` . (Nota: ci serve per aggiornare il PC, possiamo usare l'ALU principale!)
 3. **Condizione (Il trucco matematico):** Controlla se `immediate32` è divisibile per 4. Come fa l'hardware a saperlo? In binario, un numero è multiplo di 4 **solo se i suoi ultimi due bit (il bit 0 e il bit 1) sono entrambi 00** . Non serve fare divisioni, basta guardare i cavi finali!
 4. **Azione (se vero):** * Salva il valore PC+4 nel registro destinazione `rd` .
 5. Aggiorna il `PC` con la somma calcolata al punto 2.
2. Modifica il disegno del *datapath* passo dopo passo prendendo in considerazione le operazioni hardware scritte precedentemente
 3. Fatto questo attiva o disattiva i segnali di controllo dalla control unit

Indicare il contenuto in bit della word che esprime l'istruzione



3. Tipo S (Store)

Usato esclusivamente per **scrivere dati in memoria** (es. `sw`, `sh`, `sb`). Poiché qui non salviamo nulla in un registro destinazione, il campo `rd` sparisce e viene usato per fare spazio a un pezzo dell'immediato.

- **Layout (32 bit):**

- `[31-25]` : **Immediate[11:5]** (7 bit) - La parte alta del numero costante (offset).
- `[24-20]` : **rs2** (5 bit) - Il registro che contiene il dato da salvare in memoria.
- `[19-15]` : **rs1** (5 bit) - Il registro con l'indirizzo base.
- `[14-12]` : **funct3** (3 bit).
- `[11-7]` : **Immediate[4:0]** (5 bit) - La parte bassa del numero costante.
- `[6-0]` : **opcode** (7 bit).

4. Tipo B (Branch)

Usato per i **salti condizionati** (es. `beq`, `bne`, `bge`). È quasi identico al tipo S, ma l'immediato (che qui rappresenta di quante istruzioni dobbiamo saltare) è "rimescolato" per questioni di efficienza hardware.

- **Layout (32 bit):**

- `[31-25]` : **Immediate[12|10:5]** (7 bit).
- `[24-20]` : **rs2** (5 bit) - Secondo registro da comparare.
- `[19-15]` : **rs1** (5 bit) - Primo registro da comparare.
- `[14-12]` : **funct3** (3 bit) - Il tipo di comparazione (es. uguale, maggiore).
- `[11-7]` : **Immediate[4:1|11]** (5 bit).
- `[6-0]` : **opcode** (7 bit).

5. Tipo U (Upper Immediate)

Usato quando hai bisogno di caricare un **numero costante molto grande** (fino a 20 bit) in un registro (es. `lui`, `auipc`). Usa tutti i bit superiori disponibili per l'immediato.

- **Layout (32 bit):**
 - `[31-12]` : **Immediate** (20 bit) - La costante grande.
 - `[11-7]` : **rd** (5 bit) - Registro destinazione.
 - `[6-0]` : **opcode** (7 bit).

6. Tipo J (Jump)

Usato per i **salti incondizionati** (es. `jal`). Ha bisogno di un immediato grandissimo (20 bit) per poter saltare molto lontano nel programma, e di un registro destinazione per salvare l'indirizzo di ritorno.

- **Layout (32 bit):**
 - `[31-12]` : **Immediate** (20 bit) - Di quanto dobbiamo saltare (i bit sono rimescolati internamente).
 - `[11-7]` : **rd** (5 bit) - Dove salvare l'indirizzo di ritorno (PC+4).
 - `[6-0]` : **opcode** (7 bit).

Conversione in binario dei registri e delle parti immediate

Per inserire i bit nelle rispettive caselle bisogna 'convertire' i numeri dei registri, OpCode, funct7, funct3 in binario

Traduzione registri

Devi conoscere la tabella di conversione:

- `zero` = x0 (sempre `00000`)
- `ra` = x1, `sp` = x2, `gp` = x3, `tp` = x4
- `t0-t2` = x5-x7
- `s0-s1` = x8-x9

- `a0-a7` = x10-x17
- `s2-s11` = x18-x27
- `t3-t6` = x28-x31

A questo punto si trova il numero e lo si converte in binario

Traduzione OpCode e funct

L'`opcode` è sempre di **7 bit**. I campi `funct3` (3 bit) e `funct7` (7 bit) sono di solito numeri standard per istruzioni standard (che trovi nei manuali), o valori fissati dal professore se l'istruzione è inventata. Sono in esadecimale quindi basta prendere le cifre e convertirle

Traduzione Immediate

- numero semplice: traduzione classica esadecimale
- numero negativo: traduzione con complemento a due

Calcolo dei tempi di clock

Bisogna leggere attentamente i tempi di clock assegnati dall'esercizio. Il trucco è chiedersi: *Qual è il percorso più lungo (Critical Path) in sequenza che la mia nuova istruzione deve fare?*

Fatto questo si può fare la sommatoria e calcolare la tempistica. Esempio:

Tracciamo la `jalf`:

1. Legge l'istruzione dalla Memoria Istruzioni: **200 ns**.
2. Legge il registro `rs1`: **50 ns**. (*La verifica della divisibilità avviene in parallelo ed è trascurabile*).
3. L'ALU somma `rs1 + imm`: **150 ns**.
4. Il risultato passa dal MUX e va nel PC: **0 ns** (trascurabile). **Totale `jalf` = 200+50+150=400 ns**.